arXiv:2605.01941v2 [cs.DL] 11 Aug 2026

HERITRACE: a domain-agnostic framework for SHACL-driven RDF curation with provenance and change tracking

Arcangelo Massari[1][0000−0002−8420−0696]✉ and
Silvio Peroni[1,2][0000−0003−0530−4305]

[1] Research Centre for Open Scholarly Metadata, Department of Classical Philology and Italian Studies, University of Bologna, Bologna, Italy
[2] Digital Humanities Advanced Research Centre (/DH.arc), Department of Classical Philology and Italian Studies, University of Bologna, Bologna, Italy
{arcangelo.massari,silvio.peroni}@unibo.it

Abstract. HERITRACE is an open-source web application that enables users without Semantic Web expertise to curate RDF data through form-based interfaces with automatic provenance documentation and change tracking in RDF. It uses SHACL for data model definition and form generation, connects to existing SPARQL-accessible stores without data migration, and records every modification as a provenance snapshot that can be browsed and restored. HERITRACE is domain-agnostic and its configuration is layered: a minimal deployment only connects to a SPARQL store, while SHACL shapes and YAML display rules refine the data model and presentation, and IRIs are minted automatically by default or through a Python class for institution-specific schemes. This paper describes the software architecture and provides the first empirical evaluation. HERITRACE is deployed in production for the ParaText project, where classical philologists curate bibliographic data about ancient Greek exegetical traditions, and is planned as the editing interface for OpenCitations and as the curation layer for the Social Sciences and Humanities Citation Index within the GRAPHIA Horizon Europe project. Since it operates on any SPARQL-accessible store without data migration, its adoption potential extends to any domain maintaining RDF data. HERITRACE is publicly available on GitHub under the ISC license, archived on Zenodo and Software Heritage Archive, and documented for deployment with a pre-built Docker image.

Keywords: data curation · provenance · change tracking · SHACL · usability evaluation · FAIR

1 Introduction

Cultural heritage institutions increasingly use Semantic Web technologies to publish their cultural heritage collections on the Web [69]. RDF [41] and Linked

Open Data (LOD) principles [6] allow them to share metadata in machine-readable formats that are Findable, Accessible, Interoperable, and Reusable (FAIR) [73]. However, this adoption has created a tension between data quality and technical expertise. Data quality depends on domain knowledge for relationship validation, metadata enrichment, and entity disambiguation, yet the systems managing this data require proficiency in SPARQL [72] and OWL [70] that domain experts typically lack. A second gap concerns how systems represent the history of the data: tools that do provide user-friendly interfaces typically record provenance and change tracking outside RDF, in relational stores, so this metadata does not receive the same FAIR treatment as the data it documents.

Two existing cases can be briefly introduced to illustrate these gaps. The Digital Library of the Department of Classical Philology and Italian Studies (FICLIT) at the University of Bologna [1] employs OmekaS [64] to offer user-friendly interfaces but records provenance outside RDF, lacks change tracking, and requires CSV-based bulk loading for RDF import, preventing FAIR management of data and metadata. OpenCitations [57] (https://opencitations.net), an open scholarly infrastructure publishing open bibliographic metadata and citation data, implements a complete RDF representation for both its data and provenance, but requires technical expertise for data quality management, excluding librarians who could identify errors in bibliographic metadata. A solution that maintains complete provenance and change tracking within RDF while remaining usable by domain experts would be necessary for properly addressing both cases presented.

This paper presents HERITRACE, an open-source web application [44,45] formally maintained by OpenCitations, which ensures its sustainability by having HERITRACE as part of its tools and services. HERITRACE enables domain experts to curate RDF data through form-based interfaces while automatically maintaining provenance documentation and change tracking, with direct integration into existing RDF collections. A previous publication [47] introduced the system's design rationale. Since then, the system has been extended with additional functionalities: virtual properties that translate complex RDF patterns into fields that match users' mental models, configurable orphaned-entity handling, pluggable IRI minting, shape-level configuration binding that allows distinct interfaces for entities sharing the same RDF class, and context-sensitive validation through SHACL conditional constraints. Beyond these extensions, this paper contributes a description of the software architecture, a refined functional comparison with related systems, and the first empirical evaluation of HERITRACE with real users.

The rest of the paper is organised as follows. Section 2 reviews existing systems for semantic data management. Section 3 describes the HERITRACE framework in detail. Section 4 assesses HERITRACE through functional comparison, a usability study, and community adoption. Section 5 discusses the findings alongside limitations and future work.

2 Related Works

Giacomini *et al.* [22] surveyed existing systems for the management of linked open data in cultural heritage. This section builds on their survey.

OmekaS [64] provides a user-friendly web-publishing platform primarily serving museums and educational institutions. It stores both data and provenance metadata in a relational database, making it architecturally incompatible with operating directly on an existing quadstore. Moreover, it does not maintain change histories, and importing RDF data requires bulk loading through CSV conversion [5].

Semantic MediaWiki [36] integrates semantic capabilities into the MediaWiki platform, inheriting its page-level revision history for provenance and change tracking, though this information is stored in MediaWiki's relational database rather than in RDF. RDF import through the RDFIO extension [37] supports only triples, not named graphs.

ResearchSpace [52] provides diverse visualization options and records when modifications occur and which agents made them. However, it does not record what specific changes were made. Configuring the interface requires creating templates in HTML, Handlebars, and ResearchSpace-specific components rather than standard or widely adopted languages, imposing a steep learning curve on administrators. It supports direct RDF upload but requires significant programming to adapt the interface after import.

CLEF [14,22] manages digital libraries with provenance via named graphs and change tracking through GitHub synchronization, but lacks a graphical restoration interface. It supports only individual item creation, not bulk RDF import: integrating a pre-existing triplestore requires restructuring all data into CLEF's per-record named graphs, each combining entity data with provenance metadata.

Wikibase [16], the software powering Wikidata, combines user-friendliness with revision history inherited from MediaWiki, though revision data is stored in MediaWiki's relational database rather than represented in RDF. It restricts predicates to its internal Property namespace, preventing direct reuse of external OWL ontology terms and requiring conversion through tools such as O2WB [17].

Sinopia [67] generates form-based interfaces from JSON profile templates and, although primarily used for BIBFRAME workflows, supports properties from any RDF ontology. Data is stored as JSON-LD in MongoDB rather than in a triplestore, and administrative metadata recording who modified a resource and when is not represented in RDF. Previous versions of resources can be viewed but not restored. RDF data can be imported by pasting serialized RDF into a text area, but the system cannot connect to external SPARQL-accessible stores, limiting scalability for large collections.

Schímatos [74], like HERITRACE, builds web forms from SHACL shapes. It also lets developers create and edit shapes through a graphical interface, a capability HERITRACE delegates to file-based configuration. However, Schímatos manages neither provenance nor change tracking. Moreover, it provides no presentation layer to abstract the data model: entity references appear as IRIs

unless `rdfs:label` triples are added to the data, since display names cannot be configured from the graph, and reified relationships are surfaced as stored rather than abstracted into the fields a curator expects.

3 The HERITRACE Framework

The design of HERITRACE is guided by five requirements derived from the comparison criteria introduced by Giacomini *et al.* [22] and a survey of OpenCitations personnel needs for making its data modifiable by domain experts, such as librarians, while retaining the existing backend technologies.

From Giacomini *et al.*, three criteria are adopted: (R1) user-friendliness for end users, (R2) user-friendliness for administrators, and (R3) provenance management. Two criteria are refined: "data management integration", which mentions versioning without defining specific capabilities, becomes (R4) change tracking, requiring that modifications are recorded and versions can be restored; "reusability", which was too broad, becomes (R5) direct RDF import, requiring connection to pre-existing RDF collections without data migration. R3 and R4 are evaluated specifically as RDF-based capabilities: provenance and change tracking metadata should receive the same FAIR-compliant treatment as the data they document, rather than being stored in relational databases outside the Linked Open Data ecosystem.

These requirements are further justified by OpenCitations' needs [57]. Its collections are natively stored in RDF: the OpenCitations Meta collection [54] contains over 129 million bibliographic entities, and the OpenCitations Index collection [53] over 2.4 billion citations. At this scale, restructuring the data is impractical, making direct RDF import (R5) a prerequisite. Provenance management (R3) and change tracking (R4) are particularly relevant because OpenCitations aggregates data (as of 2 May 2026) from five sources – Crossref [27], DataCite [7], the NIH Open Citation Collection [29], the Japan Link Center [31], and OpenAIRE [40] – and another source, i.e. OpenAlex [61], is used for alignment purposes [63]. Tracking which source contributed each datum, when, and recording any subsequent corrections is necessary for data verifiability and reliability.

Regarding the development methodology, test-driven development [3] guided the implementation. The test suite produced during development also includes integration tests that exercise the system against real triplestore instances, testing state combinations that unit tests cannot reach. A continuous integration pipeline [28] executes the full test suite on every commit via GitHub Actions across various Python versions. Releases follow semantic versioning [60] and change logs are generated from Conventional Commits [11]. Machine-readable software metadata is provided through a CodeMeta [10] descriptor, and per-file licensing information is made machine-readable through the REUSE Specification [21]; compliance is enforced in continuous integration.

3.1 Software Architecture

HERITRACE implements a layered architecture [62] with presentation, business, persistence, and database layers (Fig. 1), processing each request through these layers sequentially. In the business layer, Access control verifies editing permissions, the RDF manager coordinates entity operations, and the Validator checks data against SHACL [35] shapes. In the persistence layer, the Query builder translates these operations into SPARQL queries for the underlying stores.

The database layer contains two SPARQL-accessible stores: the Semantic datastore holds entity data (supporting both triplestores and quadstores) and the Provenance datastore holds versioning data (requiring a quadstore for named graph-based snapshots). The two stores can be configured as separate databases or a single shared instance. A Redis [65] state store manages concurrent edit locks. Users authenticate through ORCID [24] OAuth 2.0, with an allowlist of ORCID identifiers restricting editing access to authorized personnel.

This architecture keeps each layer independent: the database layer operates with any SPARQL-accessible store, from Virtuoso [18] to QLever [2], and the presentation layer could be replaced with a framework for building multi-platform applications like Flutter [23] without modifying the business logic.

3.2 Domain-Agnostic Data Editing

The system automatically discovers entities from the connected triplestore based on their RDF class declarations. The editing view (Fig. 2) renders metadata fields through appropriate input controls: text fields, multiline text areas, date selectors, dropdown menus for controlled vocabularies, and tag inputs for keywords.

Form generation relies on two complementary configuration layers. SHACL shapes define the data model and drive widget selection: datatype declarations set which control appears (e.g. `xsd:date` produces a calendar, `xsd:gYear` a numeric field), cardinality constraints (`sh:minCount`, `sh:maxCount`) mark fields as required or repeatable, `sh:in` and `sh:or` constraints render as dropdown menus. Conditional constraints (`sh:condition`) enable context-sensitive validation [34], e.g. enforcing DOI-specific patterns when the identifier scheme is DOI. Listing 1.1 shows a shape that produces the identifier section in Fig. 2.

YAML files [4] govern presentation aspects outside SHACL's scope. SPARQL queries compute composite labels from the graph structure, for instance, assembling "Volume 3 of Nature" from a volume number and a journal title stored as separate triples. The YAML layer also controls which properties are visible and in what order, and allows per-field widget type overrides that replace the datatype-derived default (e.g. text area, date picker). Autocomplete can be configured for each form field, including the minimum character threshold before suggestions appear and whether the search targets entities of the same type or their parent resources, so that, for instance, users searching by an identifier value retrieve the bibliographic resource rather than the identifier entity itself.

When the order of values matters, the system relies on intermediate proxy entities to maintain sequence information. For example, the author's ordering in

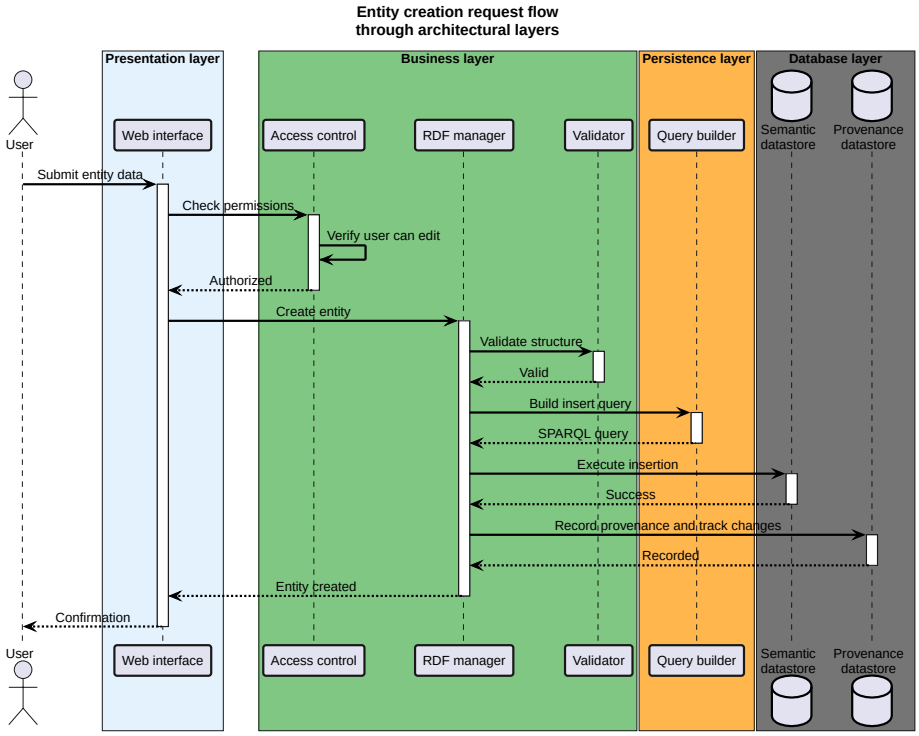

Fig. 1. Entity creation request flow through HERITRACE architectural layers

OpenCitations data uses `pro:RoleInTime` instances, as defined in the Publishing Roles Ontology (PRO) [58], as proxies: a configurable predicate chains these proxy entities so that each one points to the next. The interface exposes this ordering through drag-and-drop handles that update the chain when the user rearranges items.

The same YAML layer configures duplicate detection for each entity type by specifying which properties to compare through boolean logic. For example, a person entity can be configured to flag duplicates that share the same full name, or the same combination of given name and family name, or the same identifier.

Each YAML configuration entry is bound to data either by RDF class or by SHACL shape. Shape-level binding allows distinct presentation configurations for entities that share the same class but conform to different shapes: for example, two shapes targeting `fabio:JournalIssue` [56], one for regular journal issues and one for special issues that additionally expose a title field, can each define a separate set of visible properties.

The SHACL and YAML configuration layers are independent and entirely optional. With no configuration at all, the system falls back to free-form property

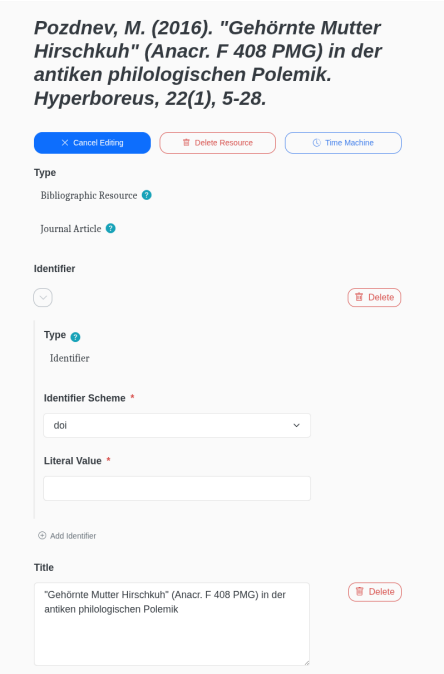

Fig. 2. Editing interface showing metadata fields and input controls

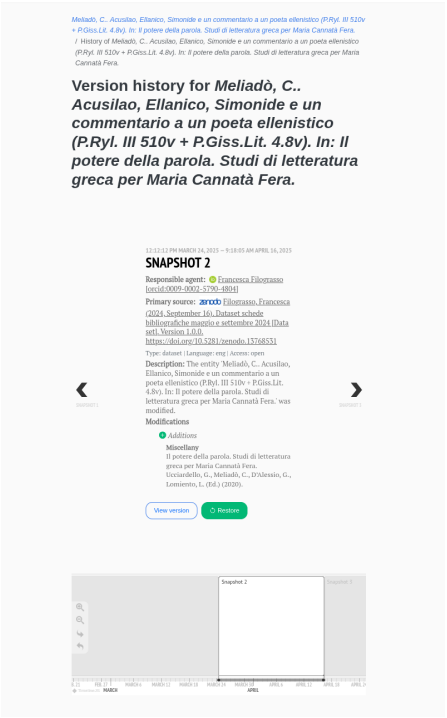

Fig. 3. Time Machine interface with version history and restoration controls

entry with display names auto-generated from URIs. Adding SHACL alone introduces structured forms with validation constraints while still deriving labels from URIs. Adding YAML alone keeps free-form entry but provides human-readable labels and controls property visibility and ordering. When both layers are present, SHACL governs form structure and validation while YAML refines the presentation. When a SHACL shape is defined for an entity type, only properties declared in that shape are available for creation and editing, ensuring data integrity with respect to the data model.

The YAML layer also supports virtual properties: fields that do not correspond to direct predicates in the data model but, when populated, automatically create and configure intermediate entities behind the scenes. Their purpose is to translate complex data models into interaction patterns aligned with users' mental models. Ontologies sometimes reify relationships as first-class entities to attach additional attributes. For instance, OpenCitations adopts the Citation Typing Ontology (CiTO) [56] to model citations as instances of a dedicated class linking a citing entity to a cited entity through separate properties. A user's mental model of this relationship is simply "I want to cite this paper", not "I want to create a Citation entity that links my article to that paper". A virtual property

```
schema:JournalArticleIdentifierShape
  a sh:NodeShape ;
  sh:targetClass datacite:Identifier ;
  sh:property [
    sh:path datacite:usesIdentifierScheme ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:in ( datacite:doi ) ;
  ] ;
  sh:property [
    sh:path literal:hasLiteralValue ;
    sh:datatype xsd:string ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
  ] ;
  sh:property [
    sh:path literal:hasLiteralValue ;
    sh:datatype xsd:string ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:condition [
      sh:path datacite:usesIdentifierScheme ;
      sh:hasValue datacite:doi ;
    ] ;
    sh:pattern "^10\\.\\d{4,9}/[-._;()/:A-Z0-9]+$" ;
  ] .
```

Listing 1.1. SHACL shape for journal article identifiers. The `sh:in` constraint generates a dropdown menu, `sh:minCount` marks fields as required, and `sh:condition` activates validation rules only when a specific identifier scheme is selected (the example includes only DOI for brevity, but a production shape would define distinct patterns for each scheme).

labeled "cites" lets users select the target resource from a form field as though the relationship were direct; the system then creates the intermediate entity (i.e. the citation), assigns it the correct type (the class `cito:Citation`), and links it to both the current resource and the selected target (via the object properties `cito:hasCitingEntity` and `cito:hasCitedEntity`, respectively), without exposing this structure to the user. Each virtual property definition targets a specific SHACL shape, from which the system derives how to create and link the intermediate entity.

When editing operations sever relationships, the system applies configurable strategies for the affected entities. Orphaned entities that are no longer referenced by any other resource and proxy entities whose parent relationship has been removed can be handled by prompting the user for a decision, automatically deleting the entity, or retaining it in the dataset, depending on the deployment configuration.

Finally, creating new entities requires minting new IRIs. A built-in generator produces IRIs by appending a universally unique identifier (UUID) to a base namespace out of the box, so a basic deployment needs no code for this. Institutions that need their own naming conventions can provide them through a pluggable Python class.

Adapting HERITRACE to a new domain is configuration-driven: SHACL shapes and YAML display rules govern the data model and presentation, with no changes to the application code; the only optional code is a Python class for custom IRI schemes. At a minimum, deploying HERITRACE on a new collection requires only connecting it to a SPARQL endpoint and launching the application through the provided Docker image.

3.3 Provenance and Change Tracking

Provenance tracking is automatic and mandatory: every creation, modification, deletion, and merge generates provenance records without requiring any explicit user action. The system implements snapshot-based versioning using the method described in the OpenCitations Data Model (OCDM) [13,49]. Each entity state is captured as a `prov:Entity` snapshot connected to the entity via `prov:specializationOf`, recording temporal boundaries, attribution, primary source, and predecessor chain through standard PROV Ontology [38] properties. In addition, each snapshot stores a SPARQL `INSERT`/`DELETE` delta via the OpenCitations Ontology data property `oco:hasUpdateQuery` [55], capturing the exact triples changed. Every operation is non-destructive: changes produce new snapshots rather than overwriting existing data or provenance records.

The Time Machine interface of HERITRACE (Fig. 3) presents the modification history of each entity as a chronological sequence of snapshots, displaying timestamps, responsible agents, primary sources, and change descriptions. Restoration replays the recorded update queries, reconstructing the entity together with the resources that the same tracked operations modified, and is itself recorded as a new snapshot. Deleted entities receive a final snapshot with a `prov:invalidatedAtTime` value; the Time Vault interface surfaces these entities, identifiable by the presence of both a generation time and an invalidation time, and allows them to be restored.

Merging entities is also non-destructive: it generates a new snapshot for the surviving entity recording the incorporated properties, while the absorbed entity's final snapshot is invalidated with a `prov:invalidatedAtTime` value. Provenance records connect the absorbed entity's snapshot to the surviving entity through `prov:wasDerivedFrom`.

To materialize past versions of entities, HERITRACE reuses the Time Agnostic Library [48], a Python library for performing temporal SPARQL queries live on any SPARQL-compliant triplestore. When HERITRACE is deployed on a pre-existing RDF collection, the configuration accepts baseline values: a default primary source and the creation timestamp of the imported data, establishing a starting point for subsequent change tracking.

4 System Assessment and Adoption

4.1 Functional Comparison

Table 1 summarizes the comparison of the systems reviewed in Section 2 against the five requirements (R1–R5) introduced in Section 3. According to our analysis, no existing system supports RDF-based change tracking (R4). HERITRACE, to the best of our knowledge, is the first RDF editing tool to record modifications as versioned snapshots natively in RDF. Beyond this novel capability, HERITRACE is the only system that simultaneously satisfies all five requirements.

Table 1. Comparison of semantic data management systems against requirements R1–R5

System	User-friendly	Admin-friendly	RDF prov. mgmt	RDF change track.	Direct RDF imp.
OmekaS	✓	✓			
Semantic MediaWiki	✓	✓			
ResearchSpace	✓				✓
CLEF	✓	✓	✓		
Wikibase	✓	✓			
Sinopia	✓	✓			✓
Schímatos		✓			✓
HERITRACE	✓	✓	✓	✓	✓

4.2 Usability Study

Prior to the formal evaluation, guerrilla testing [50] was conducted during the first production deployment of HERITRACE to surface initial issues and requirements from a real use case. In particular, we have deployed HERITRACE in ParaText [20], an Italian inter-institutional research project on ancient Greek exegetical traditions whose bibliographical database requires domain-specific classifications that generalist resources such as the Année Philologique [9] do not represent. Classical philologists without Semantic Web expertise curate this data through HERITRACE, providing a realistic setting for identifying usability issues during routine scholarly work. Informal evaluations with these scholars identified interface problems that were iteratively addressed before a new controlled study, described below.

Building on these preliminary findings, such a formal study assessed two usability dimensions: whether domain experts without Semantic Web expertise can effectively curate RDF data through form-based interfaces, and whether technical staff with varying levels of SHACL experience can configure the system for new domains. A quantitative and qualitative evaluation of HERITRACE

was conducted with two user populations: 9 end users (domain experts in bibliographic metadata) and 10 technicians (technical staff with Semantic Web knowledge). Among end users, 88.9% had no prior HERITRACE experience. Among technicians, 90% had no prior HERITRACE experience and 40% had no prior SHACL experience. Participants completed self-guided sessions of up to 60 minutes using Docker packages with pre-configured HERITRACE instances on a 100-article subset of OpenCitations Meta [46]. This setting, where participants worked independently in their own environments, maximizes ecological validity [33].

End users completed four metadata management tasks: (1) editing an existing publication (adding an author and reordering the author sequence), (2) merging duplicate author entities, (3) restoring a previous version through the HERITRACE Time Machine, and (4) creating a new publication from scratch, including authors, identifiers, and hierarchical containment relationships (i.e. from article to journal). Technicians completed two configuration tasks: (1) adding a new abstract property to the HERITRACE data model through SHACL shapes and (2) configuring YAML display rules for that property.

Data collection employed screen and voice recordings, the System Usability Scale (SUS) [8], and written reflections. Quantitative analysis tracked task completion, duration, error severity, and SUS scores. Qualitative analysis implemented grounded theory [12], a method that allows categories to emerge from the data rather than imposing predefined ones, building a theory grounded in participants' actual experiences. All evaluation materials have been published on Zenodo [43].

Figure 4 presents task completion distributions. End users achieved success rates of 78% for editing, 100% for merging, 78% for version restoration, and 67% for creating publications. The merge task demonstrated the highest success rate, with all 9 participants completing it. The two incomplete editing cases were timeouts rather than blocks: one participant browsed all records to find the publication due to the lack of a search feature; the other placed the new author in the wrong position, undid it through the Time Machine, and struggled with reordering. In version restoration, one participant misunderstood the feature and manually reverted the change instead of using the Time Machine, while the other was blocked by a bug that prevented the restore button from appearing. The create publication task yielded 6 complete and 3 partial outcomes. This task required building a containment hierarchy through nested collapsible sections (article, issue, volume, journal, and journal identifier), and the partial completions stemmed from participants losing track of previously entered data when outer sections collapsed. No participants abandoned any task due to insurmountable obstacles. Technicians achieved 90% success for both SHACL validation and YAML display configuration. One partial completion occurred due to misreading the task requirements (setting the property as required instead of optional); one timeout occurred due to placing the configuration in the wrong YAML section.

Figure 5 presents SUS score distributions. SUS is treated as a unidimensional measure following [39]. End users produced a mean SUS score of 78.9, placing

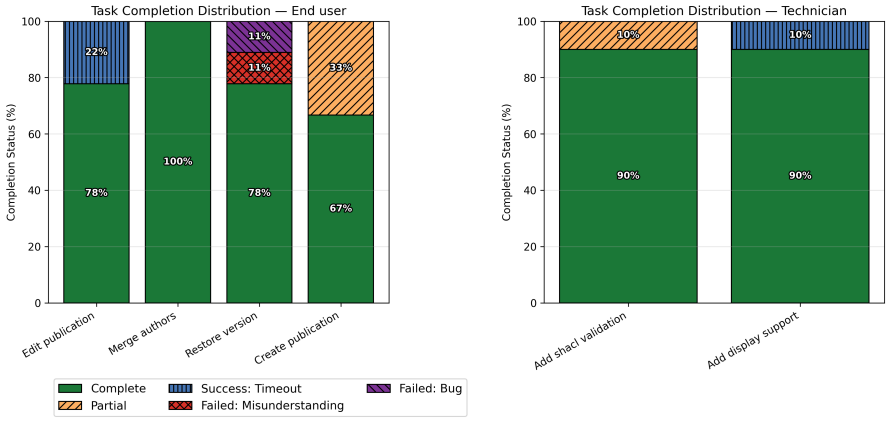

Fig. 4. Task completion status distribution by user type and task

usability in the above-average range [66]. Technicians produced a mean of 83.8, exceeding the excellent threshold of 80.3.

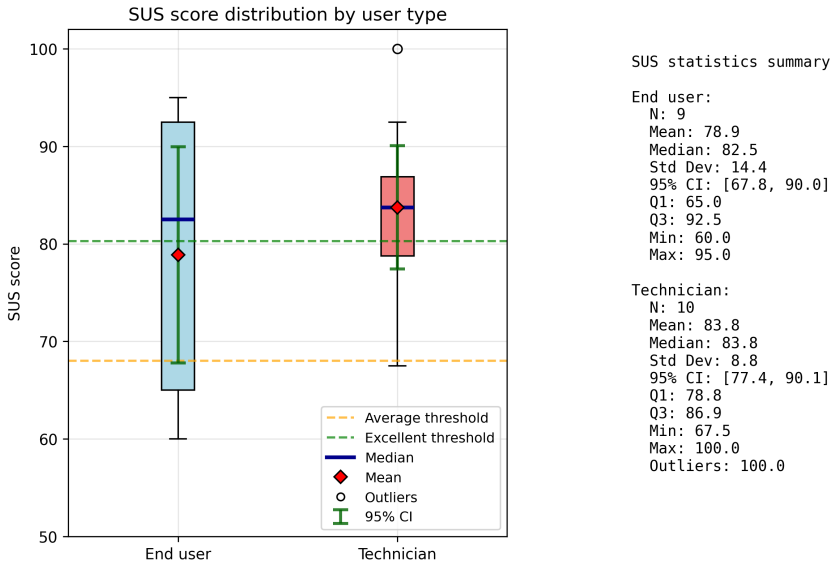

Fig. 5. SUS score distributions with benchmark thresholds for average (68) and excellent (80.3) usability [66]

Regarding the grounded theory analysis, among end users, the strongest positive responses concerned merge and duplicate management, the HERITRACE Time Machine as a safety net reducing anxiety about mistakes, and autocomplete for entity disambiguation. The most frequent negative categories were visual design problems with nested entity creation, where collapsible sections concealed previously entered data, and missing search functionality, reported by six of nine participants.

Among technicians, all participants employed pattern replication: locating existing SHACL or YAML configurations, copying them, and adapting them. This proved effective regardless of prior SHACL experience. Semantic reasoning and immediate visual feedback through automatic hot-reloading complemented this strategy. Substantive friction arose from the file-based configuration paradigm, and one technician questioned whether continuous technical support for configuration changes represents a sustainable model.

4.3 Current and Potential Community Adoption

The ParaText deployment described in Section 4.2 represents the first production adoption of HERITRACE. HERITRACE is also planned as the editing interface for OpenCitations. Separately, OpenCitations contributes to GRAPHIA [19], a Horizon Europe project that aims to build a Knowledge Graph for the Social Sciences and Humanities (SSH), consolidating data scattered across multiple organizations. Within this consortium, OpenCitations is helping develop the SSH Citation Index, and HERITRACE will be tested as a possible tool for its curation layer.

Since the system operates on any SPARQL-accessible store without requiring data migration, its adoption potential extends beyond cultural heritage and scholarly metadata to any domain that maintains RDF data. More broadly, by abstracting RDF complexity behind form-based interfaces, HERITRACE lowers the barrier to adopting Semantic Web technologies.

5 Discussion and Conclusion

The usability study provided supporting evidence that both domain experts and technicians can work effectively with HERITRACE, while also exposing friction points that warrant refinement.

The end users in the controlled study were bibliographic metadata experts. This follows from the demonstration dataset, not from the tool: HERITRACE has no bibliography-specific logic, since every field a curator sees is generated from the SHACL shapes and YAML display rules. The evaluated tasks therefore exercise a domain-independent interaction model. However, a design argument is not evidence of usability. Thus, confirming that the model serves domain experts in other fields requires cross-domain evaluation that is left to future work.

Search functionality was deliberately excluded from HERITRACE, since searching RDF datasets, like visualizing and browsing resources, presents independent complexity. In OpenCitations, dedicated tools already address each

problem separately: OSCAR [26] provides configurable text-based search over SPARQL endpoints, while LUCINDA [25] offers customizable resource visualization and browsing. HERITRACE concentrates on the editing problem. Currently, the three systems operate independently; connecting them to enable direct transitions from search results or resource browsing to the editing interface is a priority for future work.

Once users reach the editing interface, nested entity workflows introduce further friction. HERITRACE allows creating nested entities inline: when editing a publication, users can create the containing issue, volume, journal, and journal identifier within the same form, i.e. four nesting levels rendered as collapsible sections. The evaluation revealed two problems with this approach. First, collapsing a previously completed nested entity hides its content, forcing users to reopen sections to verify their input. Second, displaying all nesting levels within a single view produces cognitive overload. Two directions are under consideration: dialog-based sub-entity creation, isolating each nested entity in a separate modal window or tab so users only see the relevant context, and linearization, flattening the visual hierarchy so the containment structure remains transparent without nested collapsible sections.

On the other hand, the file-based configuration paradigm proved effective for technicians with Semantic Web expertise but raised sustainability concerns. Domain experts and technicians operate as distinct roles, so when the data model evolves (e.g. adding a class or modifying a property) the experts depend on technicians to update the SHACL and YAML configuration, a back-and-forth that can undermine the system's usability gains. This separation is not unique to HERITRACE: OmekaS, Semantic MediaWiki, ResearchSpace, and CLEF all require technical personnel at least for deployment, so eliminating the role entirely is unrealistic. A graphical configuration interface abstracting the underlying SHACL would help by exposing the supported constraints, validating fields immediately, and preventing the syntax-error crashes, while direct SHACL editing would remain for advanced users who need the full vocabulary.

Apart from the evaluation findings, the system's architectural scope also presents constraints. HERITRACE currently handles one triplestore at a time, but infrastructures such as OpenCitations manage interdependent collections served by different triplestores, where modifying an entity in one collection can affect references to it in another. Supporting such scenarios would require declaring dependency rules between collections, so that a modification in one triplestore automatically triggers the corresponding updates in the dependent ones, coordinated through atomic transactions to guarantee consistency.

The system also adopts a single-user editing model: Redis-based edit locks prevent concurrent modifications to the same entity rather than resolving conflicts. Real-time collaborative editing would require mechanisms for resolving concurrent RDF modifications. The provenance model already provides per-change attribution and version history, offering a foundation for future extension, but implementing such mechanisms exceeds the current scope.

The ordering mechanism described in Section 3.2 currently requires intermediate proxy entities chained through a designated predicate. This covers ontologies like the Publishing Roles Ontology (PRO) but not alternative ordering strategies such as `rdf:List` or explicit numeric rank properties. Supporting multiple ordering paradigms would broaden the range of ontologies it can accommodate without custom adaptation.

A related constraint is that the system operates exclusively on RDF data, yet relational databases remain the most widely adopted data management systems [15], and many cultural heritage institutions maintain their collections in relational formats [68]. Keeping both representations synchronized requires bidirectional transformation. The RDB to RDF Mapping Language (R2RML) [71] standardizes the forward direction, but inverting these mappings to propagate changes from a semantic editor back to the relational source remains an open problem. Preliminary work on R2RML mapping inversion [42] demonstrates that reconstruction is feasible when the mapping retains enough information to recover the original column values.

Beyond the data layer, user authentication also constrains deployability. HERITRACE authenticates users via ORCID OAuth 2.0, which provides both login and a persistent identifier for provenance attribution. This choice is appropriate for the academic contexts of both case studies, but ORCID adoption remains driven by journal submission requirements, with rates ranging from 17% in visual and performing arts to 93% in biological sciences among U.S. academic researchers [59]. Curators, archivists, and librarians who are not also academic researchers are unlikely to hold ORCID identifiers. Alternative identifier registries relevant to the GLAM sector, such as ISNI [30] and VIAF [51], assign persistent identifiers but do not function as identity providers, since there is no OAuth flow for either. Extending HERITRACE to non-academic GLAM contexts would therefore require decoupling authentication from identifier assignment: an institutional identity provider for login, with persistent identifiers such as ISNI or VIAF URIs associated through a separate configuration step.

A further constraint concerns the configuration layer itself, as its representation is tool-specific. HERITRACE expresses presentation rules in YAML. Expressing them in RDF, for instance through YAML-LD [32], would turn the display configuration into a queryable data model reusable by other tools, extending to the configuration layer the FAIR treatment already applied to the data and its provenance.

The limitations discussed above define the boundaries of the current system rather than of the approach itself. The evaluation results suggest that usability and FAIR compliance in RDF curation need not be mutually exclusive: form-based interfaces driven by declarative constraints can make RDF data editable by domain experts without Semantic Web expertise. Thus, in its present form, HERITRACE can already be adopted in contexts where an RDF triplestore has been set up for data management, reducing the cognitive effort for domain experts to add, modify, and remove RDF data while maintaining provenance records and change history.

Resource Availability Statement: HERITRACE is available on GitHub, archived on Zenodo [44] and Software Heritage Archive [45] under the ISC license, registered on Wikidata (Q139571902), with Docker images on Docker Hub and GitHub Container Registry. Installation and usage documentation is available at https://opencitations.github.io/heritrace/. Evaluation data and materials discussed in this paper are on Zenodo [43] (CC BY 4.0).

Acknowledgments. The authors thank Francesca Filograsso[0009−0002−5790−4804] and Camillo Neri[0000−0002−5041−4143] for their feedback during the development of HERITRACE within the ParaText project, and Bianca Gualandi[0000−0001−8202−8493] for her assistance in preparing the informed consent documentation for the usability study. This work was funded by the European Union's Horizon Europe programme, Grant Agreement No. 101188018 (GRAPHIA).

Disclosure of Interests. AM is a software developer at OpenCitations and contributed to the development of several components of its current software infrastructure. SP is the Director of OpenCitations.

Declaration of Use of Generative AI. No generative AI systems were used to design the study, generate or analyse data, produce results, or draft the scientific content of this manuscript. AI-assisted tools were used only for spell-checking and minor grammar suggestions on the English text.

References

1. ADLab - Laboratorio Analogico Digitale, /DH.arc - Digital Humanities Advanced Research Centre: FICLIT Digital Library (2022), https://dl.ficlit.unibo.it/s/lib/page/home
2. Bast, H., Buchhold, B.: QLever: A Query Engine for Efficient SPARQL+Text Search. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management. pp. 647–656. CIKM '17, Association for Computing Machinery, New York, NY, USA (2017). https://doi.org/10.1145/3132847.3132921
3. Beck, K.: Test-Driven Development: By Example. The Addison-Wesley Signature Series, Addison-Wesley, Boston (2003)
4. Ben-Kiki, O., Evans, C., Ingerson, B.: Yaml ain't markup language (yaml™) version 1.1 (2005), https://yaml.org/spec/1.1/
5. Berthereau, D.: CSV Import (2024), swh:1:snp:1ae6e996991c5a188bb7f0bb2df6c8f7a14bf4a0
6. Bizer, C., Heath, T., Berners-Lee, T.: Linked Data - The Story So Far:. International Journal on Semantic Web and Information Systems **5**(3), 1–22 (Jul 2009). https://doi.org/10.4018/jswis.2009081901
7. Brase, J., Sens, I., Lautenschlager, M.: The Tenth Anniversary of Assigning DOI Names to Scientific Data and a Five Year History of DataCite. D-Lib Magazine **21**(1/2) (2015). https://doi.org/10.1045/january2015-brase
8. Brooke, J.: SUS: A 'Quick and Dirty' Usability Scale. In: Jordan, P.W., Thomas, B., McClelland, I.L., Weerdmeester, B. (eds.) Usability Evaluation In Industry, pp. 207–212. CRC Press (Jun 1996). https://doi.org/10.1201/9781498710411-35

9. Clayman, D.L.: The Digitization of the Année Philologique. Revue de philologie, de littérature et d'histoire anciennes **XCV**(1), 147–153 (2021). https://doi.org/10.3917/e.phil.951.0147
10. The CodeMeta project (2024), https://codemeta.github.io/
11. Conventional commits 1.0.0 (2019), https://www.conventionalcommits.org/en/v1.0.0/
12. Corbin, J., Strauss, A.: Basics of Qualitative Research (3rd Ed.): Techniques and Procedures for Developing Grounded Theory. SAGE Publications, Inc., 2455 Teller Road, Thousand Oaks California 91320 United States (2008). https://doi.org/10.4135/9781452230153
13. Daquino, M., Peroni, S., Shotton, D., Colavizza, G., Ghavimi, B., Lauscher, A., Mayr, P., Romanello, M., Zumstein, P.: The OpenCitations Data Model. In: Pan, J.Z., Tamma, V., d'Amato, C., Janowicz, K., Fu, B., Polleres, A., Seneviratne, O., Kagal, L. (eds.) The Semantic Web – ISWC 2020, vol. 12507, pp. 447–463. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-62466-8_28
14. Daquino, M., Wigham, M., Daga, E., Giagnolini, L., Tomasi, F.: CLEF. A Linked Open Data Native System for Crowdsourcing. Journal on Computing and Cultural Heritage **16**(3), 1–17 (2023). https://doi.org/10.1145/3594721
15. DB-Engines: Db-engines ranking of database management systems (2026), https://db-engines.com/en/ranking
16. Diefenbach, D., Wilde, M.D., Alipio, S.: Wikibase as an Infrastructure for Knowledge Graphs: The EU Knowledge Graph. In: Hotho, A., Blomqvist, E., Dietze, S., Fokoue, A., Ding, Y., Barnaghi, P., Haller, A., Dragoni, M., Alani, H. (eds.) The Semantic Web – ISWC 2021, vol. 12922, pp. 631–647. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-88361-4_37
17. Dobriy, D., Polleres, A.: O2WB: A tool enabling ontology reuse in Wikibase. In: Proceedings of the 12th Knowledge Capture Conference 2023. pp. 101–104. ACM, Pensacola FL USA (2023). https://doi.org/10.1145/3587259.3627568
18. Erling, O., Mikhailov, I.: RDF Support in the Virtuoso DBMS. In: Pellegrini, T., Auer, S., Tochtermann, K., Schaffert, S. (eds.) Networked Knowledge – Networked Media: Integrating Knowledge Management, New Media Technologies and Semantic Systems, pp. 7–24. Springer, Berlin, Heidelberg (2009). https://doi.org/10.1007/978-3-642-02184-8_2
19. European Commission: GRAPHIA (2025), https://cordis.europa.eu/project/id/101188018
20. Filograsso, F., Massari, A., Neri, C., Peroni, S.: HERITRACE in action: The ParaText project as a case study for semantic data management in classical philology. arXiv (2025). https://doi.org/10.48550/arXiv.2508.15556
21. Free Software Foundation Europe: REUSE specification – version 3.3 (Nov 2024), https://reuse.software/spec-3.3/
22. Giacomini, S., Daquino, M., Tomasi, F., Fintoni, L.A.: CLEF 2.0. Solutions for Native Linked Data Cataloguing of Italian Digital Cultural Heritage. JLIS.it **16**(1), 108–126 (Jan 2025). https://doi.org/10.36253/jlis.it-611
23. Google: Flutter (2026), https://archive.softwareheritage.org/swh:1:snp:4afa84110351da69a1fa7574b23296fcafc585ad;origin=https://github.com/flutter/flutter
24. Haak, L.L., Fenner, M., Paglione, L., Pentz, E., Ratner, H.: ORCID: A system to uniquely identify researchers. Learned Publishing **25**(4), 259–264 (2012). https://doi.org/10.1087/20120404
25. Heibi, I.: opencitations/lucinda. Zenodo (Oct 2025). https://doi.org/10.5281/ZENODO.17476884

26. Heibi, I., Peroni, S., Shotton, D.: Enabling text search on SPARQL endpoints through OSCAR. Data Science **2**(1-2), 205–227 (Nov 2019). https://doi.org/10.3233/DS-190016
27. Hendricks, G., Tkaczyk, D., Lin, J., Feeney, P.: Crossref: The sustainable source of community-owned scholarly metadata. Quantitative Science Studies **1**(1), 414–427 (2020). https://doi.org/10.1162/qss_a_00022
28. Humble, J., Farley, D.G.: Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. A Martin Fowler Signature Book, Addison-Wesley, Upper Saddle River, NJ (2011)
29. Hutchins, B.I., Baker, K.L., Davis, M.T., Diwersy, M.A., Haque, E., Harriman, R.M., Hoppe, T.A., Leber, S.A., Renaud, T.M., Santangelo, G.M.: The NIH Open Citation Collection: A public access, broad coverage resource. PLOS Biology **17**(10), e3000385 (2019). https://doi.org/10.1371/journal.pbio.3000385
30. ISO: Information and documentation — International standard name identifier (ISNI). Standard ISO 27729:2024, International Organization for Standardization (2024)
31. Japan Link Center: Japan Link Center (JaLC) (2026), https://japanlinkcenter.org
32. Kellogg, G., Scherbakov, A., Polli, R.: YAML-LD 1.0. W3C Editor's Draft, W3C (2026), https://w3c.github.io/yaml-ld/
33. Kieffer, S., Sangiorgi, U.B., Vanderdonckt, J.: ECOVAL: A Framework for Increasing the Ecological Validity in Usability Testing. In: 2015 48th Hawaii International Conference on System Sciences. pp. 452–461. IEEE, HI, USA (Jan 2015). https://doi.org/10.1109/HICSS.2015.61
34. Knublauch, H., Allemang, D., Steyskal, S.: SHACL Advanced Features. W3C Working Group Note, W3C (Jun 2017), https://www.w3.org/TR/shacl-af/
35. Knublauch, H., Kontokostas, D.: Shapes Constraint Language (SHACL). W3C Recommendation, W3C (Jul 2017), https://www.w3.org/TR/shacl/
36. Krötzsch, M., Vrandečić, D., Völkel, M.: Semantic MediaWiki. In: Hutchison, D., Kanade, T., Kittler, J., Kleinberg, J.M., Mattern, F., Mitchell, J.C., Naor, M., Nierstrasz, O., Pandu Rangan, C., Steffen, B., Sudan, M., Terzopoulos, D., Tygar, D., Vardi, M.Y., Weikum, G., Cruz, I., Decker, S., Allemang, D., Preist, C., Schwabe, D., Mika, P., Uschold, M., Aroyo, L.M. (eds.) The Semantic Web - ISWC 2006, vol. 4273, pp. 935–942. Springer Berlin Heidelberg, Berlin, Heidelberg (2006). https://doi.org/10.1007/11926078_68
37. Lampa, S., Willighagen, E., Kohonen, P., King, A., Vrandečić, D., Grafström, R., Spjuth, O.: RDFIO: Extending Semantic MediaWiki for interoperable biomedical data management. Journal of Biomedical Semantics **8**(1), 35 (2017). https://doi.org/10.1186/s13326-017-0136-y
38. Lebo, T., Sahoo, S., McGuinness, D.: PROV-O: The PROV Ontology (2013), http://www.w3.org/TR/2013/REC-prov-o-20130430/
39. Lewis, J., Sauro, J.: Revisiting the factor structure of the system usability scale. Journal of Usability Studies **12**(4), 183–192 (Aug 2017)
40. Manghi, P., Bolikowski, L., Manola, N., Schirrwagen, J., Smith, T.: OpenAIREplus: The European Scholarly Communication Data Infrastructure. D-Lib Magazine **18**(9/10) (2012). https://doi.org/10.1045/september2012-manghi
41. Manola, F., Miller, E.: RDF Primer (Feb 2004), http://www.w3.org/TR/2004/REC-rdf-primer-20040210/
42. Massari, A.: arcangelo7/knowledge-graphs-inversion: V1.2.0. Zenodo (Feb 2026). https://doi.org/10.5281/ZENODO.18773042
43. Massari, A.: HERITRACE user testing: Packages and results (Apr 2026). https://doi.org/10.5281/zenodo.19822398

44. Massari, A.: opencitations/heritrace v3.0.0. Zenodo (2026). https://doi.org/10.5281/ZENODO.18642856
45. Massari, A.: opencitations/heritrace v3.0.0. Software Heritage Archive (2026), https://archive.softwareheritage.org/swh:1:snp:1d58ffefc1133ebc84a7aebb75cb14c3521e98b4;origin=https://github.com/opencitations/heritrace
46. Massari, A., Mariani, F., Heibi, I., Peroni, S., Shotton, D.: OpenCitations Meta. Quantitative Science Studies **5**(1), 50–75 (2024). https://doi.org/10.1162/qss_a_00292
47. Massari, A., Peroni, S.: HERITRACE: A User-Friendly Semantic Data Editor with Change Tracking and Provenance Management for Cultural Heritage Institutions. Umanistica Digitale **9**(20), 317–340 (Jul 2025). https://doi.org/10.6092/ISSN.2532-8816/21218
48. Massari, A., Peroni, S.: Time travel for knowledge graphs: Live queries over RDF change histories (2026), https://arxiv.org/abs/2210.02534
49. Massari, A., Peroni, S., Tomasi, F., Heibi, I.: Representing provenance and track changes of cultural heritage metadata in RDF: A survey of existing approaches. Digital Scholarship in the Humanities **41**(Supplement 1), i196–i212 (2026). https://doi.org/10.1093/llc/fqaf076
50. Nielsen, J.: Usability Engineering. Academic Press, Boston (1993)
51. OCLC: VIAF: Virtual International Authority File (2026), https://viaf.org/
52. Oldman, D., Tanase, D.: Reshaping the Knowledge Graph by Connecting Researchers, Data and Practices in ResearchSpace. In: Vrandečić, D., Bontcheva, K., Suárez-Figueroa, M.C., Presutti, V., Celino, I., Sabou, M., Kaffee, L.A., Simperl, E. (eds.) The Semantic Web – ISWC 2018, vol. 11137, pp. 325–340. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-030-00668-6_20
53. OpenCitations: OpenCitations Index CSV dataset of all the citation data (2026). https://doi.org/10.6084/m9.figshare.24356626
54. OpenCitations: OpenCitations Meta CSV dataset of all bibliographic metadata (2026). https://doi.org/10.5281/zenodo.18324537
55. Peroni, S., Shotton, D., Vitali, F.: A Document-inspired Way for Tracking Changes of RDF Data. In: Hollink, L., Darányi, S., Peñuela, A., Kontopoulos, E. (eds.) Detection, Representation and Management of Concept Drift in Linked Open Data. pp. 26–33. CEUR Workshop Proceedings, Bologna (2016), http://ceur-ws.org/Vol-1799/Drift-a-LOD2016_paper_4.pdf
56. Peroni, S., Shotton, D.: FaBiO and CiTO: Ontologies for describing bibliographic resources and citations. Journal of Web Semantics **17**, 33–43 (2012). https://doi.org/10.1016/j.websem.2012.08.001
57. Peroni, S., Shotton, D.: OpenCitations, an infrastructure organization for open scholarship. Quantitative Science Studies **1**(1), 428–444 (Feb 2020). https://doi.org/10.1162/qss_a_00023
58. Peroni, S., Shotton, D., Vitali, F.: Scholarly publishing and linked data: Describing roles, statuses, temporal and contextual extents. In: Proceedings of the 8th International Conference on Semantic Systems. pp. 9–16. ACM, Graz Austria (2012). https://doi.org/10.1145/2362499.2362502
59. Porter, S.R., Umbach, P.D., Willis, C.: Understanding ORCID adoption among academic researchers. Scientometrics **130**, 2783–2797 (2025). https://doi.org/10.1007/s11192-025-05300-7
60. Preston-Werner, T.: Semantic versioning 2.0.0 (2013), https://semver.org/
61. Priem, J., Piwowar, H., Orr, R.: OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts (2022). https://doi.org/10.48550/arXiv.2205.01833

62. Richards, M., Ford, N.: Fundamentals of Software Architecture: An Engineering Approach. O'Reilly, Beijing Boston Farnham Sebastopol Tokyo (2020)
63. Rizzetto, E., Peroni, S.: Mapping bibliographic metadata collections: the case of OpenCitations Meta and OpenAlex. In: Bernasconi, E., Mannocci, A., Poggi, A., Salatino, A., Silvello, G. (eds.) Proceedings of the 20th Conference on Information and Research Science Connecting to Digital and Library Science (IRCDL 2024). CEUR Workshop Proceedings, vol. 3643, pp. 155–168. CEUR-WS (2024), https://ceur-ws.org/Vol-3643/#paper15
64. Rueff, B.: Dealing with post-excavation data: The Omeka S TiMMA web-database. Peer Community Journal **4**, e5 (2024). https://doi.org/10.24072/pcjournal.366, https://peercommunityjournal.org/articles/10.24072/pcjournal.366/
65. Sanfilippo, S.: Redis (2009), https://archive.softwareheritage.org/swh:1:snp:67e81e2c6975cdc61546a106749482760b430f92;origin=https://github.com/redis/redis
66. Sauro, J.: Quantifying the User Experience: Practical Statistics for User Research. Morgan Kaufmann (2016)
67. Schreur, P.E.: Sinopia: A new linked-data editing environment designed for libraries. In: Garoufallou, E., Fallucchi, F., William De Luca, E. (eds.) Metadata and Semantic Research. Communications in Computer and Information Science, vol. 1057, pp. 425–430. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-36599-8_39
68. Stefanova, S., Risch, T.: Scalable reconstruction of RDF-archived relational databases. In: Proceedings of the Fifth Workshop on Semantic Web Information Management. SWIM '13, Association for Computing Machinery, New York, NY, USA (2013). https://doi.org/10.1145/2484712.2484717, https://doi.org/10.1145/2484712.2484717
69. Tamper, M.: From Text to Knowledge: Methods, Tools, and Applications for Digital Humanities Based on Linked Data. Ph.D. thesis, Aalto University (2023)
70. W3C: OWL 2 Web Ontology Language Document Overview (Second Edition) (2012), https://www.w3.org/TR/owl2-overview/
71. W3C: R2RML: RDB to RDF Mapping Language (2012), https://www.w3.org/TR/r2rml/
72. W3C: SPARQL 1.1 Query Language (2013), https://www.w3.org/TR/sparql11-query/
73. Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.W., da Silva Santos, L.B., Bourne, P.E., Bouwman, J., Brookes, A.J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C.T., Finkers, R., Gonzalez-Beltran, A., Gray, A.J., Groth, P., Goble, C., Grethe, J.S., Heringa, J., 't Hoen, P.A., Hooft, R., Kuhn, T., Kok, R., Kok, J., Lusher, S.J., Martone, M.E., Mons, A., Packer, A.L., Persson, B., Rocca-Serra, P., Roos, M., van Schaik, R., Sansone, S.A., Schultes, E., Sengstag, T., Slater, T., Strawn, G., Swertz, M.A., Thompson, M., van der Lei, J., van Mulligen, E., Velterop, J., Waagmeester, A., Wittenburg, P., Wolstencroft, K., Zhao, J., Mons, B.: The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data **3**, 1–9 (Mar 2016). https://doi.org/10.1038/sdata.2016.18
74. Wright, J., Rodríguez Méndez, S.J., Haller, A., Taylor, K., Omran, P.G.: Schímatos: A shacl-based web-form generator for knowledge graph editing. In: Pan, J.Z., Tamma, V., d'Amato, C., Janowicz, K., Fu, B., Polleres, A., Seneviratne, O., Kagal, L. (eds.) The Semantic Web – ISWC 2020. pp. 65–80. Springer International Publishing, Cham (2020)